

Website Security & Risk Assessment

lately.la · Lately Technologies, Inc.

CONFIDENTIAL

Prepared: June 22, 2026

Engagement: Authorized internal assessment — passive / read-only phase

Method: Black-box, anonymous. No data altered, stored, or exfiltrated.

This report explains, in plain language, what we found and why it matters — with the technical detail underneath each item. It is written so a non-technical owner can grasp the risk and a developer can act on it.

Security findings are factual; legal/compliance notes are informational and not legal advice.

1. Executive summary

We performed an authorized, non-disruptive security review of the Lately website. Nothing was broken, no accounts were used, and no customer data was downloaded — we only confirmed *that* problems exist, then stopped. Here is the bottom line.

The headline, in one breath

The website keeps customer data safe using a single layer of permission rules — and several of those rules are missing. **Right now, real customer information (names, email addresses, and phone numbers from reviews) can be read by anyone on the internet, with no login.**

Importantly, **no master password or secret key was leaked**. The problem is misconfigured permissions, which is fixable with configuration changes — not a rebuild.

Overall risk posture

Severity	Count	What it covers
CRITICAL	1	Live exposure of customer personal information (PII)
HIGH	3	Exposed device tokens, all discount codes, spoofable email
MEDIUM	5	Staff data, account-lookup oracle, open CORS, test code in prod, missing headers
LOW / INFO	4	Config disclosure, weak token storage, no read rate-limit, hygiene debt
NEEDS DEEPER TEST	6+	Payment tampering, privilege escalation, cross-user access — require the authorized account-based window

The good news (what is NOT broken)

No master key was exposed. The all-powerful database key that would bypass every protection is not in the code — so there is no instant total-takeover path.

Outsiders cannot get into the admin panel. The admin login holds, and admin data does not leak to anonymous visitors.

Payment-processor credentials (Stripe/Square) are not exposed to anonymous users, and core financial tables return nothing without a login.

Encryption (HTTPS) is modern and correct. No source code or environment secrets are leaking.

Because the real problems are configuration-level, the urgent ones are fixable in hours, not weeks.

What to do first

1. **Fix the customer-data exposure today.** It is live and involves real California residents' contact information, which can carry legal breach-notification duties (see §7).
2. **Brief the team / counsel** on that exposure before anything else.
3. Then close the remaining quick wins (email, headers, open CORS, test code) and schedule the deeper authorized test for the items that still need a logged-in account.

2. How the site is built (and why it matters)

Lately is a single-page web app (Vite/React) hosted on Vercel. Its data lives in a service called Supabase, and — this is the key design choice — the website talks to that database **directly from the visitor's browser**, with no traditional server in between.

The filing-cabinet analogy

Normally a security guard (a backend server) stands between visitors and the filing cabinets (the database). The guard checks who you are before fetching anything.

Lately removed the guard and instead put a **lock on each individual drawer** — these locks are called Row-Level Security (RLS) policies. The public “anon key” baked into the site is just the building's front-door key that everyone is meant to have; that part is normal and fine.

The problem we found: several drawers were left unlocked. Because those per-drawer locks are the *only* protection, one missing lock means that entire table of data is readable by anyone who walks in.

This is why the same root cause produces several findings below, and why the fix is “repair the locks” (configuration) rather than “rebuild the building.”

3. Confirmed findings

Each item is stated plainly, then technically, then with the realistic consequence if exploited.

CRITICAL

F1 — Customer personal information readable by anyone

In plain English. The table that stores reviews also stores customers' names, email addresses, and phone numbers — and that table can be read by anyone on the internet without logging in. About 28 records are exposed today.

Technically. GET /rest/v1/booking_reviews returns HTTP 206 with full rows including customer_name, customer_email, and customer_phone. RLS is not restricting the anonymous role, and CORS reflects any origin, so the data is readable cross-site from a malicious page as well.

If exploited

Scrapers, competitors, or scammers can harvest your customers' contact details for spam, targeted phishing, or sale. Because the data belongs to California residents, this likely qualifies as a reportable data breach with notification obligations — a legal and reputational hit, not just a technical one.

Mitigating

The exposed record count is small, and the fix is immediate: a corrected policy plus serving public reviews through a view that omits personal fields.

HIGH

F2 — Device push-notification tokens exposed

In plain English. The list of 468 device “address labels” used to send app notifications is public, and each one is linked to a specific user.

Technically. push_tokens is anon-readable: token, device_id, platform, and user_id are all returned.

If exploited

It lets an outsider enumerate and link devices to users (a privacy leak), and — worst case, if the tokens are usable — abuse or spoof the push channel to send notifications to your users.

Mitigating

Straightforward fix: restrict the table to the owning user (user_id = the logged-in user) and remove anonymous access entirely.

HIGH

F3 — Every discount code is exposed

In plain English. Anyone can read all active promo codes and exactly how much each one is worth.

Technically. promo_codes is anon-readable: code, discount_type, discount_value, usage_limit, current_usage_count, is_active.

If exploited

Direct revenue leakage — codes get shared and abused, eroding margins. If combined with the still-untested price-tampering path, discount abuse could be worse.

Mitigating

Fix: stop reading promo codes in the browser; validate and apply them server-side only (the app already has a validate-promo-code function for this).

MEDIUM

F4 — Staff details exposed

In plain English. Names, Instagram handles, and working days for 49 staff members are publicly readable.

Technically. `provider_staff` is anon-readable (name, role, instagram_handle, working_days, platform_user_id).

If exploited

Aids social engineering and is a minor privacy issue; low direct damage but unnecessary exposure.

Mitigating

Serve a public projection with display name only, or scope the table per provider.

MEDIUM

F5 — Anyone can check if an email has an account

In plain English. A function lets anyone submit an email address and learn whether it belongs to a Lately account — and get that account's internal ID.

Technically. `rpc/get_user_id_by_email` is invocable by the anonymous role (returns a user UUID for real accounts; tested only with a fake address).

If exploited

This is account enumeration (an attacker can confirm who your customers are) and, more importantly, the internal ID is exactly the “key” needed to target the currently-protected tables if any other gap is found. It also fuels convincing phishing.

Mitigating

Fix: remove anonymous (and ideally all client) permission to run this function; resolve emails only inside trusted server code.

MEDIUM

F6 — The data service accepts requests from any website

In plain English. The backend does not restrict which websites are allowed to talk to it from a browser.

Technically. CORS reflects an arbitrary Origin header (e.g. a forged attacker origin is echoed back in `access-control-allow-origin`).

If exploited

On its own it is low-impact, but it **amplifies** F1–F4: a malicious website a customer visits could read the exposed data through that customer's browser.

Mitigating

Lock the allowed origin to the Lately app domain(s).

MEDIUM**F7 — Test code is running in the live payment path**

In plain English. “Test” versions of payment functions are shipped and reachable in production alongside the real ones.

Technically. create-setup-intent-test, confirm-setup-booking-test, and an RPC get_services_with_timeslots_test are present; both Stripe publishable keys and test-mode flags are in the bundle.

If exploited

Test code paths are typically less hardened. The real concern (still to be confirmed in the authorized window) is whether test mode could be forced to complete a \$0 booking. At minimum it is a maintenance and integrity risk.

Mitigating

Remove the *-test functions and RPC from production and make Stripe mode strictly server-determined.

MEDIUM**F8 — Confirmation emails can be spoofed**

In plain English. Your domain is not configured to stop other people from sending email that looks like it comes from Lately.

Technically. No SPF record exists, and there are three conflicting DMARC records (all at p=none) — multiple records make DMARC invalid, and p=none enforces nothing.

If exploited

An attacker can send fake “your booking is confirmed” emails to your customers (phishing), damaging trust and potentially harvesting credentials or payments.

Mitigating

Quick DNS fix: publish one SPF record, collapse to a single DMARC record, and move toward an enforcing policy (quarantine).

MEDIUM**F9 — Standard browser-security protections are missing**

In plain English. The site is missing several standard HTTP headers that browsers use to block common attacks.

Technically. Only HSTS is present. No Content-Security-Policy, no frame protection (X-Frame-Options / frame-ancestors), no X-Content-Type-Options, Referrer-Policy, or Permissions-Policy.

If exploited

The booking/checkout pages can be embedded invisibly in a malicious site (clickjacking), and there is no defense-in-depth if any cross-site-scripting bug exists.

Mitigating

One-file fix via vercel.json adding the missing headers.

LOW

F10 — No rate limiting on public reads

In plain English. The API does not appear to slow down or block rapid repeated requests.

Technically. A capped 20-request burst to `/rest/v1/services` returned 20 successes and zero throttling (429s).

If exploited

Enables cheap scraping and inflated cost/availability pressure. The more serious concern — rate limiting on login and OTP/SMS — was not tested (it carries cost/lockout risk) and is flagged for the authorized window.

Mitigating

Add platform-level rate limiting / WAF in front of the public endpoints.

LOW

F11 — Login sessions stored where scripts can read them

In plain English. When a user logs in, the session is kept in browser storage that page scripts can access.

Technically. supabase-js default stores the session in `localStorage` (no `HttpOnly` protection).

If exploited

On its own it is benign, but if any cross-site-scripting bug exists, an attacker could steal a live session. It raises the stakes of every other input-handling issue.

Mitigating

Mitigated sharply once a Content-Security-Policy is in place and stored-XSS is confirmed absent.

Minor / informational

- **S2 (Low).** An owner's personal email address is hardcoded in the code (also the Mapbox account owner). Minor exposure / hygiene.
- **S3 (Info).** Both Stripe *publishable* keys (including the test key) are shipped. Publishable keys are public by design, but shipping the test key is hygiene debt tied to F7.

4. What is protected (verified good)

For balance — these were specifically checked and are currently holding:

- **No master (service-role) key in the code.** Every embedded key was decoded across all 86 code chunks; only the public anon key exists. There is no instant RLS-bypass path.
- **Admin panel resists outsiders.** All /admin routes redirect unauthenticated visitors to login; the server-side `is_admin_user()` check returns false to anonymous callers; no admin data leaked.
- **Payment-processor credential tables (Stripe/Square) are not anon-readable** — they return empty to anonymous requests today.
- **Core financial/customer tables** (bookings, profiles, payments, loyalty, redemption codes) return nothing without a login.
- **Transport security is solid** — modern TLS 1.2/1.3 only, valid certificate, HTTPS enforced.
- **No leaks of source maps, .env files, or git data** in production.
- **Storage buckets are not publicly listable.**

Why this matters for the fix

Because the confirmed exposures come from **misconfigured permissions, not a leaked key**, the remedy is policy work inside Supabase. There is no need to rotate a master secret or rebuild — you repair the locks on the specific drawers and verify the rest.

5. What still needs testing

The passive, anonymous phase is complete. The following can only be confirmed with the authorized, account-based window (they need a logged-in user, a write, or a controlled payment). This is where the remaining serious risk could live — they are not yet confirmed either way.

- **Cross-user data access (IDOR).** Can a logged-in user read another user's records — especially another provider's Stripe/Square credentials? Those tables are safe from anonymous users but their logged-in rules are unverified. **Highest-priority verification.**
- **Price/payment tampering.** The checkout calculates the charge amount in the browser. If the server does not recompute it, prices could be altered before payment. **Highest-value item to close.**
- **Privilege escalation to admin.** Can a normal logged-in user flip their own role/verified status, or perform admin-only writes (e.g. approve their own reviews, edit promo codes)? This would grant admin power without the panel.
- **Admin login strength.** The admin gate appears to be a single shared password with no multi-factor and unconfirmed rate limiting — its resistance to guessing was deliberately not tested.
- **Login / OTP / SMS abuse.** Rate limiting on authentication and text-message sending (toll-fraud risk).
- **Stored cross-site scripting** in review or bio text (requires submitting content).

6. Priorities & remediation

Priority	Issue	Fix	Effort
P0	Customer PII & broken table locks (F1–F4, F8-data)	Enable RLS on every table; remove anon/USING(true) grants except the genuine public catalog; serve reviews via a PII-free view; scope push_tokens to the user; make promo codes server-only.	Low (hrs)
P0	Verify provider Stripe/Square credential tables	Confirm logged-in policies are scoped to the owning provider. Fastest via the Supabase dashboard.	Minutes
P1	Account-lookup oracle (F5) & open CORS (F6)	Revoke anon permission on get_user_id_by_email; lock CORS to the app domain.	Low
P1	Test code in payment path (F7)	Delete *-test functions/RPC from prod; force Stripe mode server-side; confirm the server recomputes the charge amount.	Low–Med
P1	Spoofable email (F8)	Publish one SPF record; collapse to a single DMARC record; move toward an enforcing policy.	Minutes (DNS)
P2	Browser security headers (F9), rate limiting (F10)	Add CSP, frame protection, nosniff, Referrer-Policy, Permissions-Policy via vercel.json; add WAF/rate limits.	Low
P2	Compliance gaps (see §7)	Assess CCPA notification duty with counsel; add consent banner, Do-Not-Sell link, GPC handling; disclose session replay.	Med

7. Compliance & legal note

Informational only — not legal advice. Confirm specifics with qualified counsel.

Because finding F1 exposes the contact information of what are likely California residents, it may trigger **CCPA/CPRA data-breach considerations**, including potential notification duties. The exposure window matters, so fixing F1 promptly also limits that exposure. This should be reviewed with counsel quickly.

Separately, the broader privacy review found related (non-breach) compliance debt for a California business:

- No cookie/consent banner — analytics, ad pixels, and **PostHog session replay** fire before any consent.
- No “Do Not Sell or Share My Personal Information” link and no Global Privacy Control handling (required under CPRA).
- The privacy policy does not name CCPA/CPRA and does not disclose the session-replay tool that is actually running.

On data sensitivity

Beauty/wellness booking data can be considered sensitive. Session replay without input masking can record what customers type into forms — worth closing alongside the consent gaps.

8. Operational issues (beyond security)

Captured during the review and worth a separate workstream — they hit growth and conversion rather than safety:

Performance

- ~2.6 MB of JavaScript loads up front; the map library (Mapbox) is 61% of it and loads eagerly on every page even when no map is shown.
- The homepage fires roughly 134 network requests (an “N+1” pattern), making mobile load slow — which directly suppresses booking conversion.

Search visibility (SEO)

- The site renders entirely in the browser and serves identical page titles/descriptions for every URL, so shared booking links show a generic preview on iMessage, Slack, Facebook, etc.
- Unknown URLs return a “200 OK” instead of a proper 404, and the rich-result data for local search is thin; individual provider pages are not in the sitemap, so they cannot rank.

Accessibility & reliability

- A form field is missing a name/label (breaks autofill and screen readers); the booking calendar’s keyboard/screen-reader support is unverified — the area most exposed to ADA complaints.
- Cached API responses can show a “last-minute” slot that is already gone, which breaks the core promise at checkout.

Appendix — scope & conduct

- **Authorization.** Internal assessment of the company's own web application, performed at owner request. Black-box (website only), anonymous/logged-out.
- **Conduct.** Non-destructive and read-only. Reachability was proven with HTTP status codes and row counts (zero rows transferred); at most one row was sampled for column names only, with all values discarded. No personal data was stored.
- **Not done.** No accounts created, no bookings, no charges, no OTP/SMS, no writes, no admin actions, and nothing used to authenticate. Login passwords were not guessed or brute-forced.
- **Low-noise mode.** Probes were paced (2–4s, no bursts) and analytics data egress was suppressed during testing.
- **Out of scope.** Stripe, Vercel, Mapbox, PostHog, Google, and any partner systems.
- **Stack.** Vite/React SPA on Vercel; Supabase (PostgREST API, RPCs, Auth, Storage) called directly from the browser; Stripe payments; Google Workspace email.

Full technical evidence, endpoint-level detail, SQL remediation snippets, and the deferred-test plan are maintained in the working file *lately-active-findings.md*.